

Multimodal AI Models

Landscape Overview

Sources: GroqCloud Console | OpenAI Developer Docs

Vision

Audio

Image Gen

Video

Speech

Realtime

What Are Multimodal Models?

Multimodal AI models can process and generate content across multiple data types — text, images, audio, video, and more — in a single unified model.



Vision / Image

Understand & analyze images



Audio / Speech

Process voice & speech



Video

Generate & understand video



Realtime

Live audio + text streaming

Two platforms surveyed: GroqCloud & OpenAI Developer Portal

GroqCloud — Multimodal Models

console.groq.com

Model	Modalities	Category	Context
Llama 4 Scout 17B 16E	Text + Vision (Images)	Vision / Multimodal	131K
GPT OSS 120B	Text + Multilingual + Reasoning	Text / Reasoning	131K
GPT OSS 20B	Text + Multilingual + Reasoning	Text / Reasoning	131K
Whisper Large v3	Speech → Text	Speech-to-Text	—
Whisper Large v3 Turbo	Speech → Text (fast)	Speech-to-Text	—
Orpheus English (TTS)	Text → Speech	Text-to-Speech	4K
Orpheus Arabic Saudi (TTS)	Text → Speech (Arabic)	Text-to-Speech	4K
Safety GPT OSS 20B	Text + Content Safety	Moderation	131K

OpenAI — Multimodal Models (1/2)

developers.openai.com

Model	Modalities	Category	Notes
GPT-5.4 / GPT-5	Text + Vision + Audio	Frontier	Flagship model
GPT-5.4 Pro / GPT-5 Pro	Text + Vision + Audio	Frontier	Smarter responses
GPT-5 mini / GPT-5 nano	Text + Vision	Frontier	Cost-efficient
GPT-4.1 / 4.1 mini / 4.1 nano	Text + Vision	Frontier	Smartest non-reasoning
GPT-4o / GPT-4o mini	Text + Vision + Audio	Realtime / Chat	Flexible, fast
gpt-realtime-1.5 / gpt-realtime	Text + Audio (realtime)	Realtime Audio	Audio in/out
gpt-realtime-mini	Text + Audio (realtime)	Realtime Audio	Cost-efficient
gpt-audio-1.5 / gpt-audio	Text + Audio	Chat Completions	Audio in/out
gpt-audio-mini	Text + Audio	Chat Completions	Cost-efficient

OpenAI — Multimodal Models (2/2)

Model	Modalities	Category	Notes
GPT-4o Transcribe	Speech → Text	Speech-to-Text	Powered by GPT-4o
GPT-4o mini Transcribe	Speech → Text	Speech-to-Text	Cost-efficient
GPT-4o Transcribe Diarize	Speech → Text + Speakers	Transcription	Speaker ID
GPT-4o mini TTS	Text → Speech	Text-to-Speech	Powered by GPT-4o mini
TTS-1 / TTS-1 HD	Text → Speech	Text-to-Speech	Speed / Quality
Whisper	Audio → Text	Speech Recognition	General-purpose
GPT Image 1.5 / GPT Image 1	Text → Image	Image Generation	State-of-the-art
gpt-image-1-mini	Text → Image	Image Generation	Cost-efficient
DALL·E 3 / DALL·E 2	Text → Image	Image Gen (Dep.)	Previous generation
Sora 2 / Sora 2 Pro	Text → Video	Video Generation	Synced audio
GPT-4o Audio / mini Audio	Text + Audio	Audio Chat	Chat Completions

Multimodal Capability Summary

Vision / Image Understanding

5 models

Groq + OpenAI

Speech-to-Text

5 models

Groq + OpenAI

Text-to-Speech

5 models

Groq + OpenAI

Image Generation

5 models

OpenAI

Video Generation

2 models

OpenAI

Realtime Audio

5 models

OpenAI

Multimodal Text + Audio

6 models

OpenAI

Multilingual / Reasoning

4 models

Groq + OpenAI

Key Takeaways

01

OpenAI leads in modality breadth

OpenAI covers 6+ modality types: text, vision, audio, realtime, image gen, and video generation — a wider multimodal footprint than Groq.

02

Groq excels at speed + accessibility

Groq offers fast inference (500–1000 t/s) on multimodal-capable open-weight models (Llama 4 Scout, Whisper), making multimodal AI highly accessible.

03

Vision is universally supported

Both platforms provide vision-capable models (Llama 4 Scout on Groq; GPT-4o, GPT-5 family on OpenAI) for image understanding tasks.

04

Realtime audio is an OpenAI differentiator

OpenAI's Realtime models (gpt-realtime-1.5, GPT-4o Realtime) enable live bidirectional audio — a capability absent from Groq's current lineup.



Verified Model Data (2026)

Top Models Across Providers - Cost &
Performance Analysis

Model	Context	Input (\$/1M)	Output (\$/1M)
GPT-5.4 Pro	~1M	\$30	\$180
GPT-5	400K	\$1.25	\$10
o1	200K	\$15	\$60

Model	Context	Input (\$/1M)	Output (\$/1M)
Claude Opus 4.6	1M	\$5	\$25
Claude Sonnet 4.5	1M	\$3	\$15
Claude Haiku	~200K	\$0.25-1	\$1-5

Model	Context	Input (\$/1M)	Output (\$/1M)
DeepSeek Chat	164K	\$0.32	\$0.89
DeepSeek R1	128-160K	~\$0.50	~\$1.00
DeepSeek V3	~128K	~\$0.50	~\$1.00

Model	Context	Input (\$/1M)	Output (\$/1M)
Gemini 1.5 Pro	1M+	~\$3.50	~\$10
Gemini 1.5 Flash	1M	~\$0.35	~\$1
Gemini Nano	Small	Very Low	Very Low

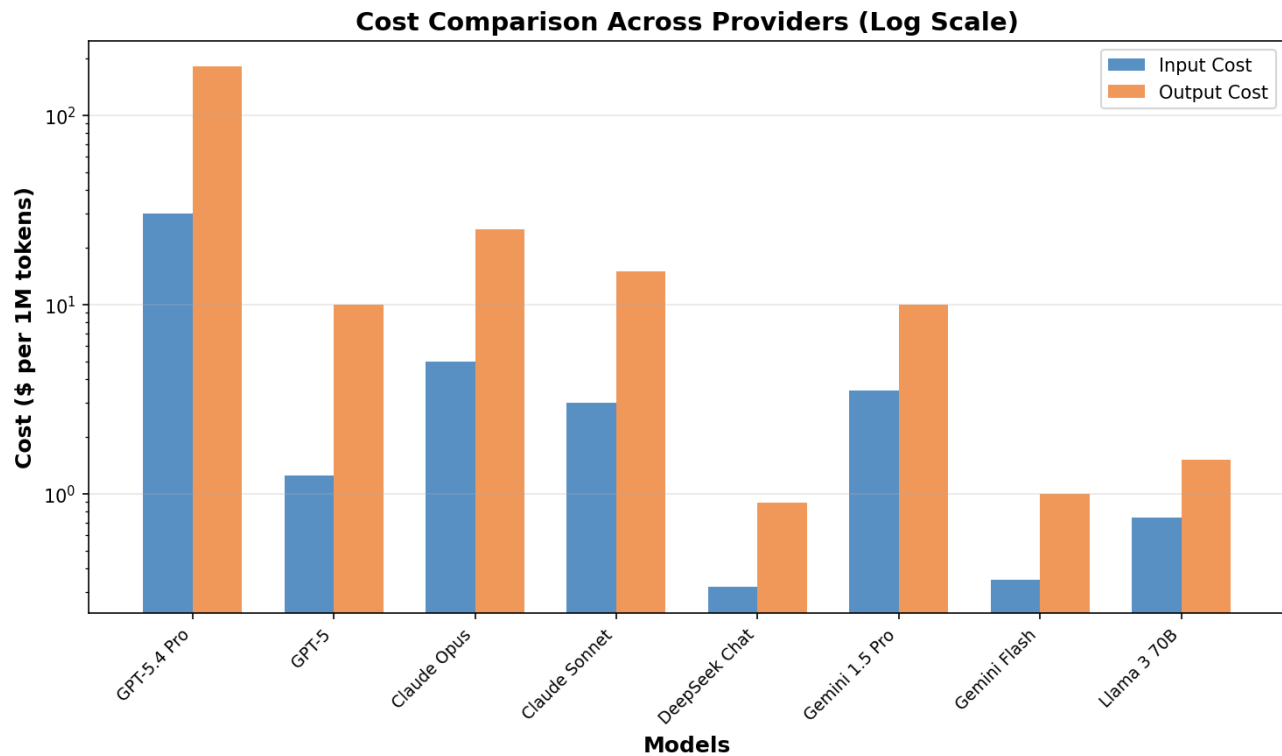
Model	Context	Input (\$/1M)	Output (\$/1M)
Llama 3 70B	32-128K	\$0.50-1	\$1-2
Llama 3 8B	~32K	~\$0.20	~\$0.40
Code Llama	16-32K	~\$0.50	~\$1

Model	Context	Input (\$/1M)	Output (\$/1M)
Mistral Large	32-128K	\$2-4	\$6-12
Mixtral 8x7B	32K	\$0.70	\$0.70
Mistral Small	~32K	~\$0.20	~\$0.60

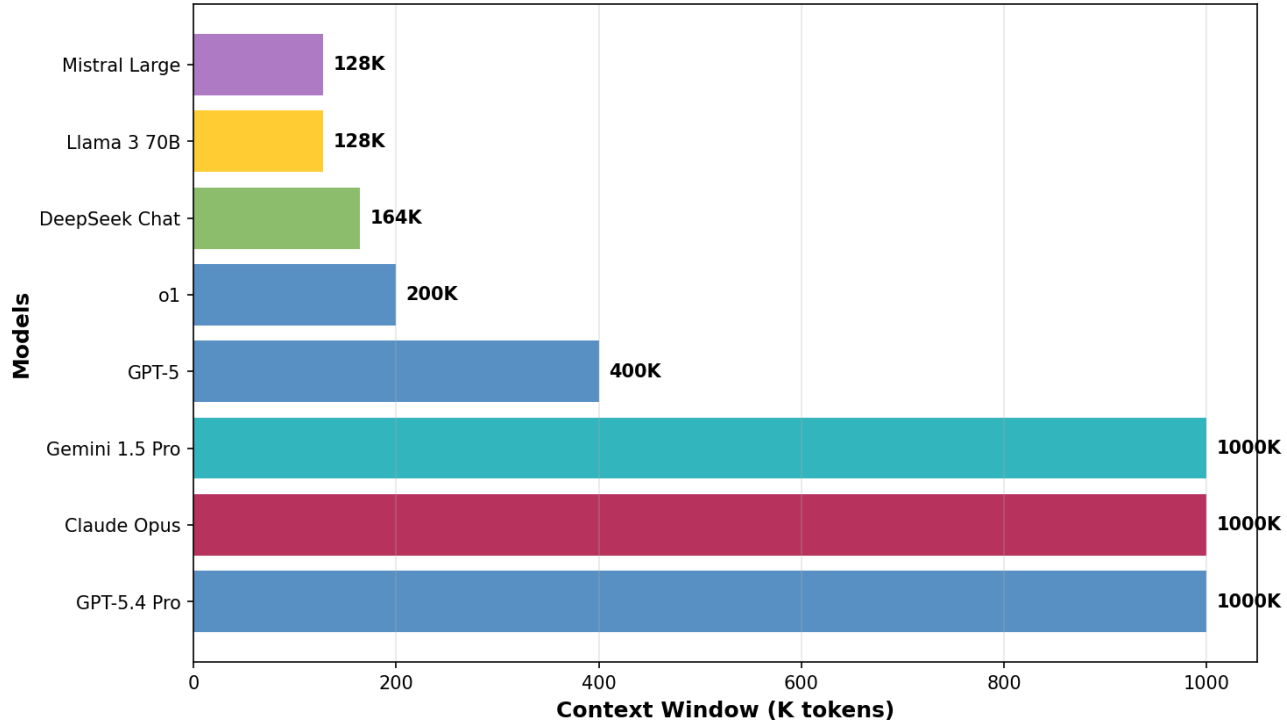
Model	Context	Input (\$/1M)	Output (\$/1M)
Grok-1.5	~128K+	~\$5	~\$15
Grok-1	~128K	~\$3	~\$10
Grok Mini	Smaller	Low	Low

Model	Context	Input (\$/1M)	Output (\$/1M)
Qwen 2.5 72B	~128K	\$1-2	\$2-4
Qwen 2	~128K	~\$0.80	~\$2
Qwen 1.5	32-64K	~\$0.30	~\$1

Provider	Advantage	Best For
DeepSeek	~50x cheaper than GPT-5.4 Pro	Most cost-effective
Claude Sonnet	Best balance of price/performance	Sweet spot for most tasks
GPT-5	Best reasoning vs cost ratio	Premium reasoning capabilities



Context Window Comparison



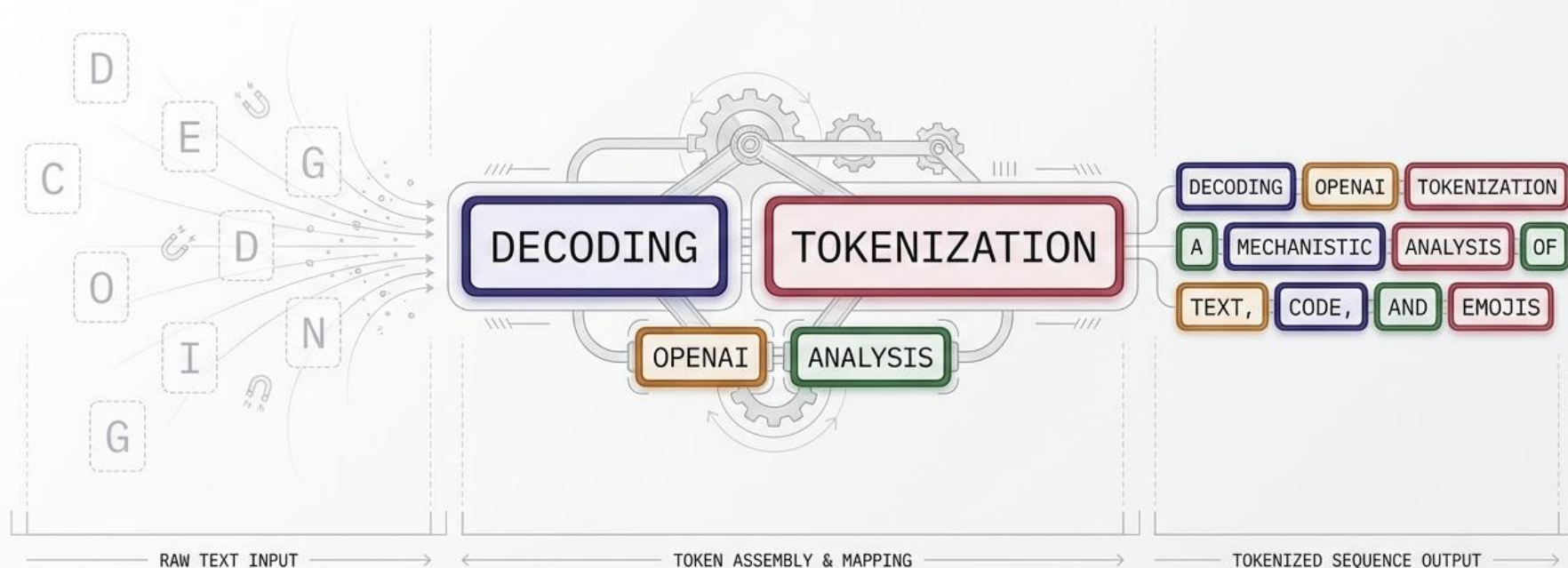
Model Decision Matrix

Use Case	Best Model	Key Advantage
Reasoning	GPT-5	Best for complex reasoning tasks
Long Documents	Claude Opus	1M context, perfect for books/reports
Cheap Scale	DeepSeek Chat	50x cheaper than premium models
Realtime Apps	Gemini Flash	Fast, cheap, 1M context
Best Balance	Claude Sonnet	Price/performance sweet spot
Code Generation	GPT-5 / Claude	Strong coding capabilities

Tier	Models	Characteristics
Premium Tier	GPT-5.4 Pro, Claude Opus	Best performance, highest cost
Balanced Tier	GPT-5, Claude Sonnet, Gemini Pro	Good performance, moderate cost
Budget Tier	DeepSeek, Gemini Flash, Llama 3	Cost-effective, good for scale

DECODING OPENAI TOKENIZATION

A MECHANISTIC ANALYSIS OF TEXT, CODE, AND EMOJIS

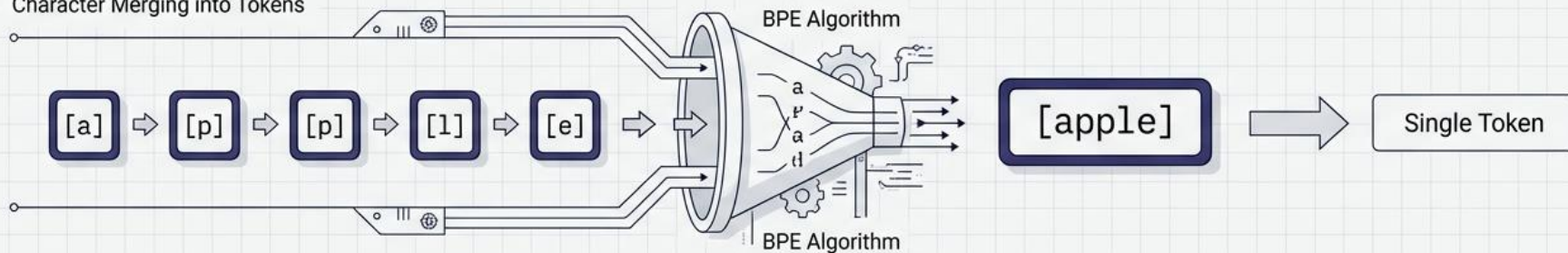


Byte Pair Encoding Mechanism

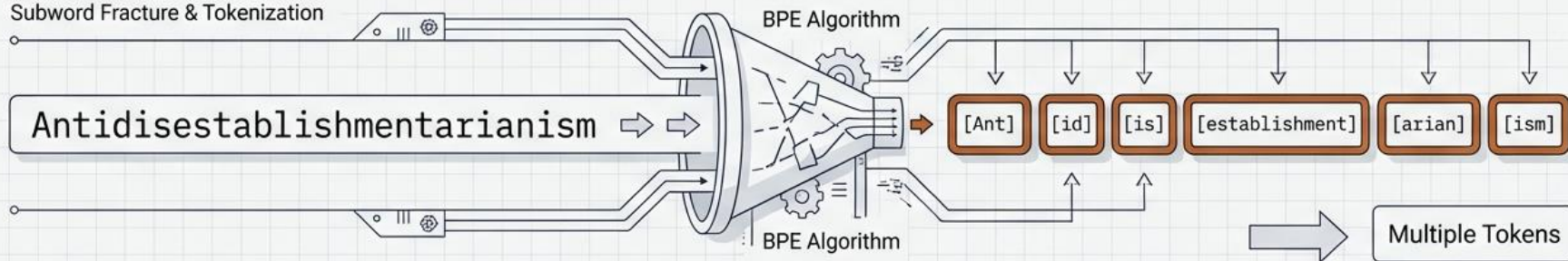
LLMs do not read words; they read subwords or characters represented by numeric IDs.

The Baseline Rule: 1,000 tokens $\approx$ 750 words.

TOP TRACK
Character Merging into Tokens

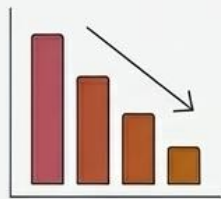


BOTTOM TRACK
Subword Fracture & Tokenization



Token Economics & Constraints

$$\text{Tokens} = \text{API Cost} + \text{Context Limits}$$



Budget Drain

Pricing is calculated per token, making hidden inefficiencies a direct drain on engineering budgets.

Capacity Limits

Danger Zone

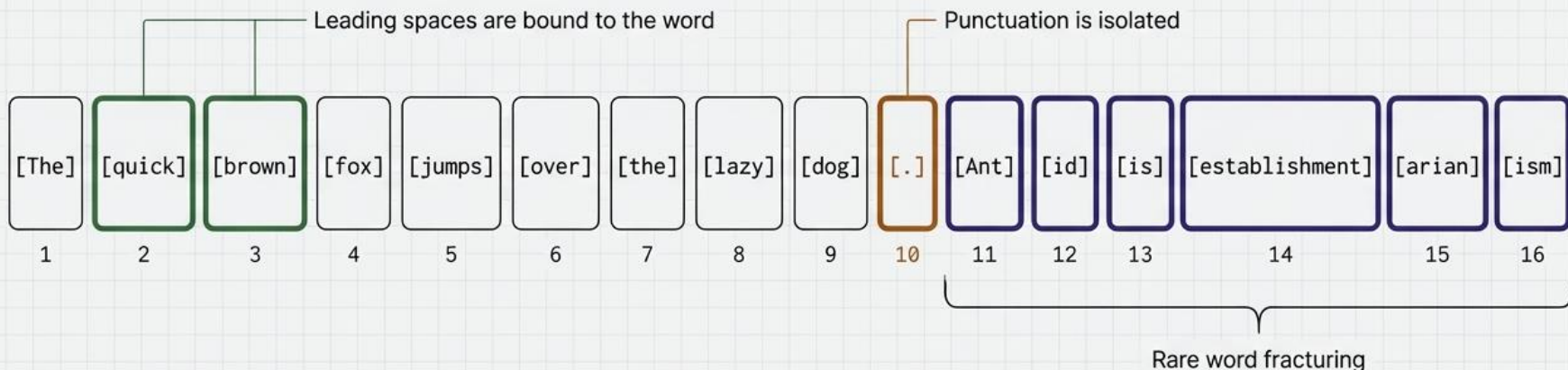


Every wasted token pushes the model closer to unexpected text truncation and context amnesia.

Case 1: The Anatomy of a String

Simple, common words map 1:1. Rare words trigger aggressive subword fracturing.

Total Count: 16 Tokens



Case 2: The Invisible Cost of Whitespace

Total Count: 13 Tokens

RAW INPUT

```
def hello():  
    print('Hi!')
```

TOKENIZED OUTPUT



The Indentation Impact

Four spaces (or tabs) act as unique, dedicated tokens rather than empty space.

Syntax Fracturing

Symbols like () : and ' are treated as individual, isolated tokens.

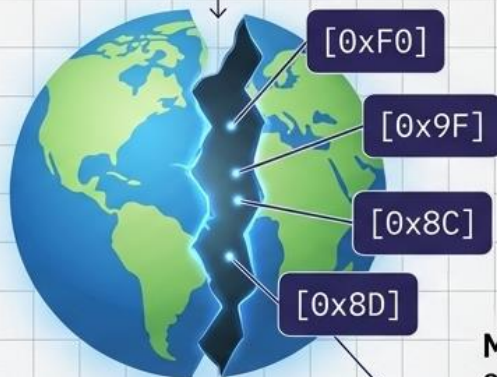
Case 3: The Emoji Tax

Input: Hello 🌐! How are you? 😊

Total Count: 11 Tokens

Proximity Rules

The space immediately preceding an emoji is inconsistently parsed—often stranded as a separate token.



Emoji Multi-Byte Explorer

Multi-Byte Composition: Complex emojis are not single characters; their multi-byte Unicode composition forces them to consume up to 4-6 tokens each.

[Hello] [] [0xF0] [0x9F] [0x8C] [0x8D] [!] [How] [are] [you] [?] [] [😊]

Case 4: The Multilingual Penalty

Tokenizers are heavily optimized for English text. Other languages require significantly more tokens to convey the exact same meaning, multiplying API costs.

English Baseline

Input:

I love apples

Output Tokens:

[I] [love] [apples]

Result: 3 Tokens

Chinese Diagnostic

Input:

我喜欢苹果

Output Tokens:

[我] [喜] [欢] [苹] [果]

Result: 5 Tokens



The Hidden Bias of Tokens

The tokenizer defaults heavily to standard English prose. Code formatting, non-Latin characters, and Unicode elements incur an immediate, invisible token penalty.

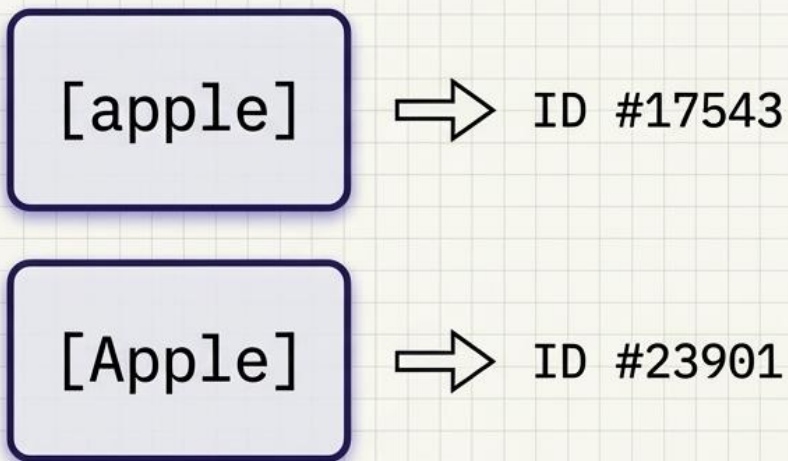
Token Efficiency Matrix

Input Type	Example	Token Count	Efficiency Grade	System Impact
Simple English	[I love apples]	3 Tokens	High	Baseline Cost
Code (Indented)	[def hello():]	13 Tokens	Medium	Context Bloat
Multilingual	[我喜欢苹果]	5 Tokens	Low	High Token Tax
Emojis	[🌍]	Up to 6 Tokens	Lowest	Extreme Token Tax

Engineering Playbook: Space & Case Sensitivity

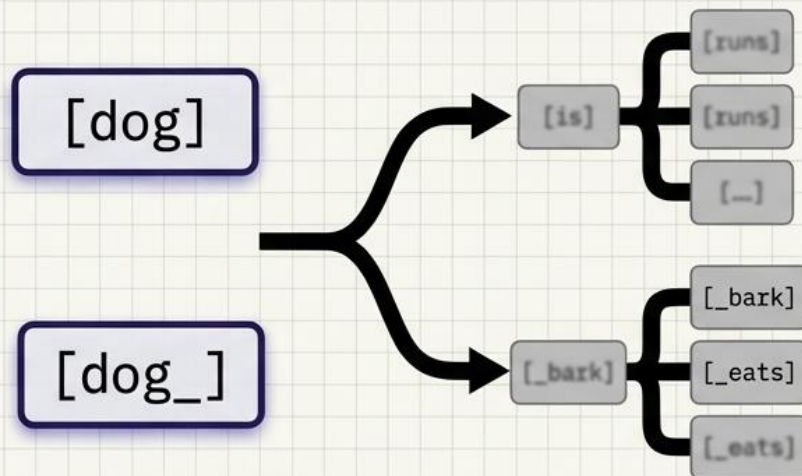
Micro-level nuances dictate prompt accuracy and model routing.

Case Sensitivity



Capitalization completely alters the token ID, changing how the model contextualizes the word.

The Trailing Space



Trailing spaces actively alter the model's entire prediction tree.

Architectural Tactics for Token Optimization

Concrete tactics for managing API limits and reducing systemic costs.

